UNIVERSIDAD PRIVADA DE TACNA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA DE SISTEMAS



EXAMEN PRÁCTICO DE LA UNIDAD II

CURSO:

SI-988 SOLUCIONES MÓVILES II

AUTOR:

2022073503 - COLQUE PONCE SERGIO ALBERTO

URL GITHUB:

https://github.com/srg-cp/examen-u2

TIPO DE EXAMEN: 1

DOCENTE:

DR. OSCAR JUAN JIMENEZ FLORES

TACNA — PERÚ

Indicaciones

- El docente le asigna un único tipo de examen (1, 2, 3 o 4). Desarrolle exclusivamente el tipo asignado y márquelo claramente en la carátula.
- Trabajará sobre la app móvil de su proyecto con los avances de los Sprints culminados en la unidad 2, y desplegada en el emulador o en un dispositivo físico propio. El login de la Unidad I debe seguir siendo funcional y la app debe mostrar ya datos de su lógica de negocio. La calificación se centra en el código cliente (Mobile App).
- La evidencia será un único archivo PDF nombrado obligatoriamente SI988-EXPRAC-U2-ApellidoNombre.pdf y subido al Aula Virtual.
- El documento debe incluir el desarrollo paso a paso de cada ítem de su tipo de examen, con capturas legibles del entorno de desarrollo, del emulador o dispositivo y de la terminal, más una breve explicación técnica de lo implementado.
- Suba todo el código fuente con los cambios implementados a su repositorio individual de GitHub, llamada examen-u2. En el README.md detalle el tipo de examen y la lista de cambios y los archivos del proyecto móvil afectados.
- El uso de herramientas de IA está permitido bajo su estricta responsabilidad. No se admitirá atribución de errores a la IA y el estudiante debe dominar y explicar técnicamente cada línea de código.
- No copie código de talleres ni de compañeros. Las clases, vistas, entidades y paquetes deben corresponder a su propio proyecto, y cada ítem se aplica a una funcionalidad real de su app, no a una pantalla de demostración aislada. Cualquier detección de plagio o código duplicado anulará la evaluación sin lugar a reclamo.
- *Al concluir su desarrollo, deberá convocar al docente para la validación en tiempo real de todos los puntos programados y pruebas ejecutadas en su computador.*

TIPO DE EXAMEN 1

Historia de usuario. *Como usuario de la app, quiero ver en el mapa dónde estoy y saber si estoy cerca de un lugar de interés, sin que la app agote la batería rápidamente ni se quede en blanco cuando el GPS falla.*

1. Separe la ubicación del resto de la app (4 puntos)

Objetivo. *La lógica de la pantalla obtiene la ubicación sin depender directamente de la biblioteca de GPS, de modo que pueda probarse sin GPS.*

Tarea 1. La ubicación se representa con una entidad propia de la app que incluye latitud, longitud y precisión en metros.

Tarea 2. Solo una clase de la capa de datos usa la biblioteca de GPS. El ViewModel la recibe desde fuera y no importa esa biblioteca.

Tarea 3. La pantalla muestra la latitud y la longitud obtenidas. Fije la posición del emulador desde sus controles extendidos.

Evidencia de la app. Captura de la pantalla con la latitud y la longitud fijadas en el emulador.
Evidencia del código. Captura de la entidad, de la clase que usa la biblioteca de GPS y de los imports del ViewModel.

2. Pida la ubicación con precisión media y apague el GPS al salir (4 puntos)

Objetivo. *La app no agota la batería pidiendo más ubicación de la que necesita.*

Tarea 1. La app no usa la máxima precisión del GPS.

Tarea 2. La ubicación se actualiza como máximo cada 30 segundos y solo si el usuario se movió 50 m o más.

Tarea 3. Si no hay lectura en 10 segundos, la app muestra «Tiempo agotado».

Tarea 4. Al salir de la pantalla del mapa, la app deja de usar el GPS y escribe «GPS detenido» en el log.

Evidencia de la app. Captura del Logcat o de la consola con «GPS detenido» justo después de salir de la pantalla.
Evidencia del código. Captura del código que cumple cada tarea, con las líneas resaltadas.

3. Muestre el radio de precisión y una alternativa sin GPS (4 puntos)

Objetivo. *El usuario ve cuán exacta es su ubicación y puede seguir usando la app aunque el GPS falle.*

Tarea 1. La pantalla muestra un mapa de OpenStreetMap con su atribución visible.

Tarea 2. La posición del usuario se dibuja como un círculo cuyo radio en metros es la precisión de la lectura, y debajo se lee «Precisión ± N m».

Tarea 3. Si no hay ubicación (GPS apagado, permiso denegado o tiempo agotado), la app muestra el motivo y permite que el usuario elija su ubicación tocando el mapa.

Evidencia de la app. Una captura con el círculo y «Precisión ± N m», y otra con el GPS del emulador apagado y la ubicación elegida en el mapa.
Evidencia del código. Captura del código que dibuja el círculo y del que ofrece la elección en el mapa.

4. Ejecute tres pruebas unitarias del ViewModel de ubicación (4 puntos)

Objetivo. *Se comprueba el ViewModel sin emulador ni GPS, reemplazando el servicio de ubicación por uno simulado.*

Tarea 1. Prueba (a). Al crear el ViewModel no se pide ninguna ubicación.

Tarea 2. Prueba (b). Con una lectura simulada en latitud -18.0066, longitud -70.2463 y precisión de 35 m, el estado expone esas coordenadas y esa precisión.

Tarea 3. Prueba (c). Con el GPS apagado simulado, el estado pasa a la elección en el mapa y la ubicación se pidió una sola vez.

Tarea 4. Ejecute las pruebas desde la terminal.

Evidencia de la terminal. Captura con el comando ejecutado y el resultado de todas las pruebas en verde (PASS).

Evidencia del código. Captura de las pruebas y del reemplazo simulado que usan.

5. Avise si el usuario está cerca de un lugar y pruébelo (4 puntos)

Objetivo. *La app avisa al usuario cuando está dentro de la zona de un lugar de interés.*

Tarea 1. En el mapa se marca un lugar de interés de su negocio con una zona de 150 m, y debajo se lee «Estás dentro de la zona de <lugar>» o «Estás fuera de la zona de <lugar>».

Tarea 2. Prueba (a). Con centro en -18.0066, -70.2463 y zona de 150 m, el punto -18.0057, -70.2463 (a unos 100 m) queda dentro y el punto -18.0039, -70.2463 (a unos 300 m) queda fuera.

Tarea 3. Prueba (b). Después de salir de la pantalla no queda ninguna escucha de ubicación activa.

Tarea 4. Ejecute las pruebas desde la terminal.

Evidencia de la app. Dos capturas del mapa con el lugar de interés, una con el usuario dentro de la zona y otra fuera.

Evidencia de la terminal. Captura con el comando ejecutado y el resultado de todas las pruebas en verde (PASS).

Evidencia del código. Captura del cálculo de la zona y de las dos pruebas.

DESARROLLO DEL EXAMEN TIPO 1

Punto 1 Separación de la ubicación

La ubicación se integra en Explorar y su mapa de la aplicación móvil Entre Huellas. La lectura del dispositivo pasa por la capa de datos, el repositorio y el caso de uso antes de llegar al ViewModel. La interfaz muestra las coordenadas recibidas. Este apartado documenta la entidad, el acceso al GPS y los imports del ViewModel; las evidencias de ejecución se incorporan a continuación.

Paso 1 Representación de la ubicación

Implementé la entidad CurrentLocation en domain/entities. Contiene latitude, longitude y accuracyMeters, que representan latitud, longitud y precisión en metros. No importa Flutter ni geolocator, por lo que puede utilizarse con lecturas simuladas en las pruebas.

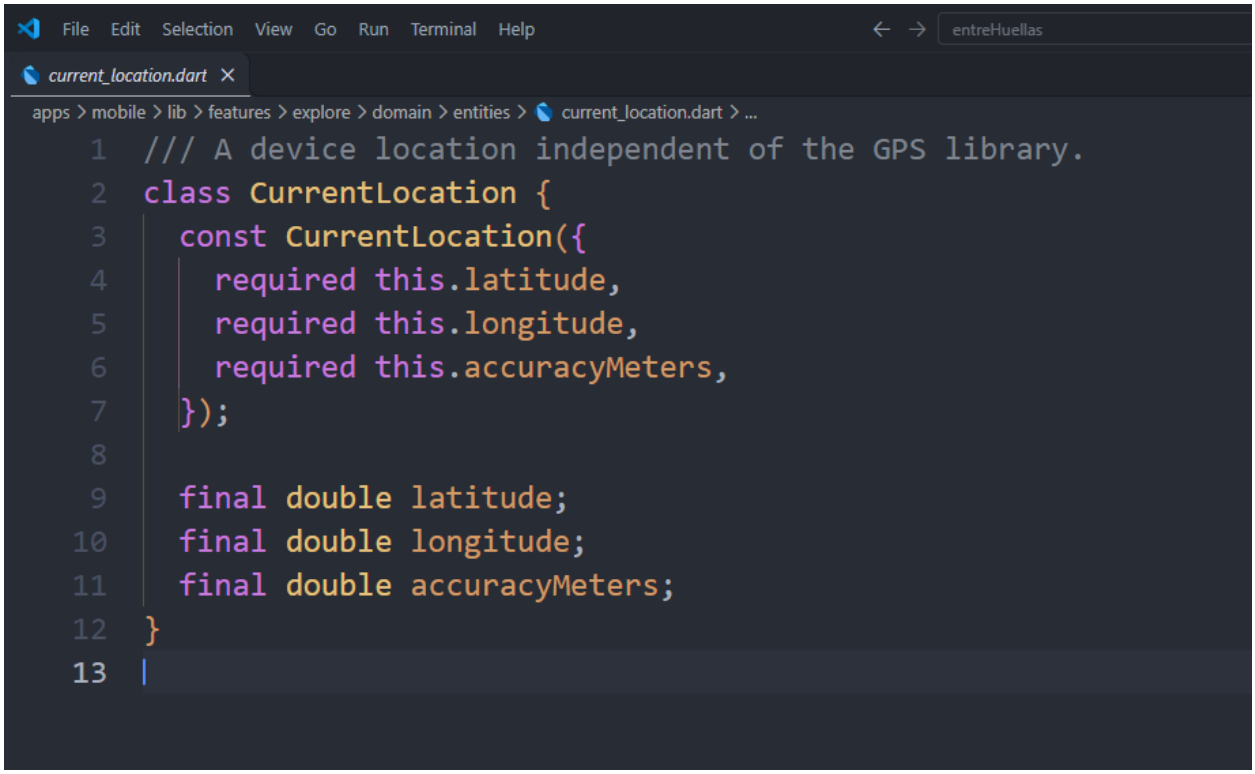


Figura 1. Entidad CurrentLocation con coordenadas y precisión en metros.

Paso 2 Acceso al GPS en la capa de datos

DeviceLocationSource es la única clase que importa geolocator. Comprueba si el servicio está habilitado, verifica o solicita el permiso y obtiene la posición con getCurrentPosition. Devuelve latitude, longitude y position.accuracy como accuracyMeters. CurrentLocationRepositoryImpl transforma esta lectura en la entidad del dominio.



Figura 2. Fuente de datos que concentra el acceso a la biblioteca GPS.

La captura corresponde al punto 1. El timeout visible de 12 segundos se ajustará a 10 segundos al desarrollar el punto 2.

Paso 3 Consumo desde el ViewModel

ExploreViewModel obtiene GetCurrentExploreLocation mediante los providers existentes. El caso de uso depende del contrato CurrentLocationRepository; su implementación se inyecta desde main. Así, la lógica de pantalla trabaja con la entidad propia y no llama directamente a geolocator.

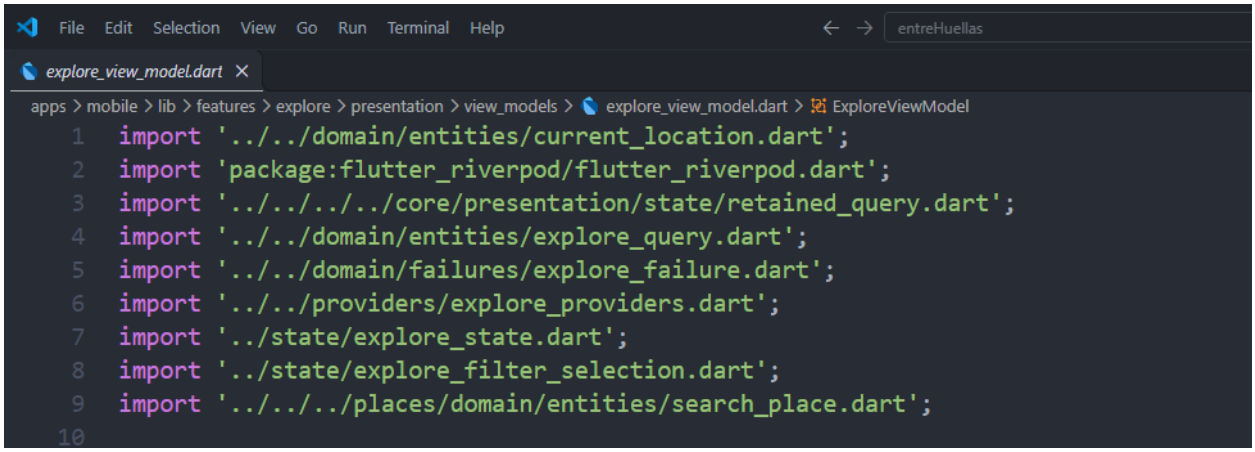


Figura 3. Imports de ExploreViewModel sin dependencia de geolocator.

ExploreState conserva la ubicación obtenida, incluida su precisión. ExploreMapScreen pasa esa entidad a ExploreCaseMap, que muestra Latitud y Longitud con seis decimales en el panel inferior cuando no hay un caso seleccionado. La lectura GPS se conserva durante el refresco y aunque falle la consulta de casos.

Comprobación local

Se ejecutaron 36 pruebas de ubicación, Explorar, filtros, mapa y arquitectura, con resultado satisfactorio, y flutter analyze sin incidencias. Estos resultados validan el código local; la captura del emulador y la captura de la aplicación documentan la configuración y la lectura nativas.

Paso 4 Configuración de la posición del emulador

En Extended Controls del emulador Pixel 9a abrí Location y fijé la latitud -18.0066 y longitud -70.2463, correspondientes a Tacna. La confirmación de la ventana indica que la ubicación fue establecida y el interruptor Enable GPS signal permanece activado.

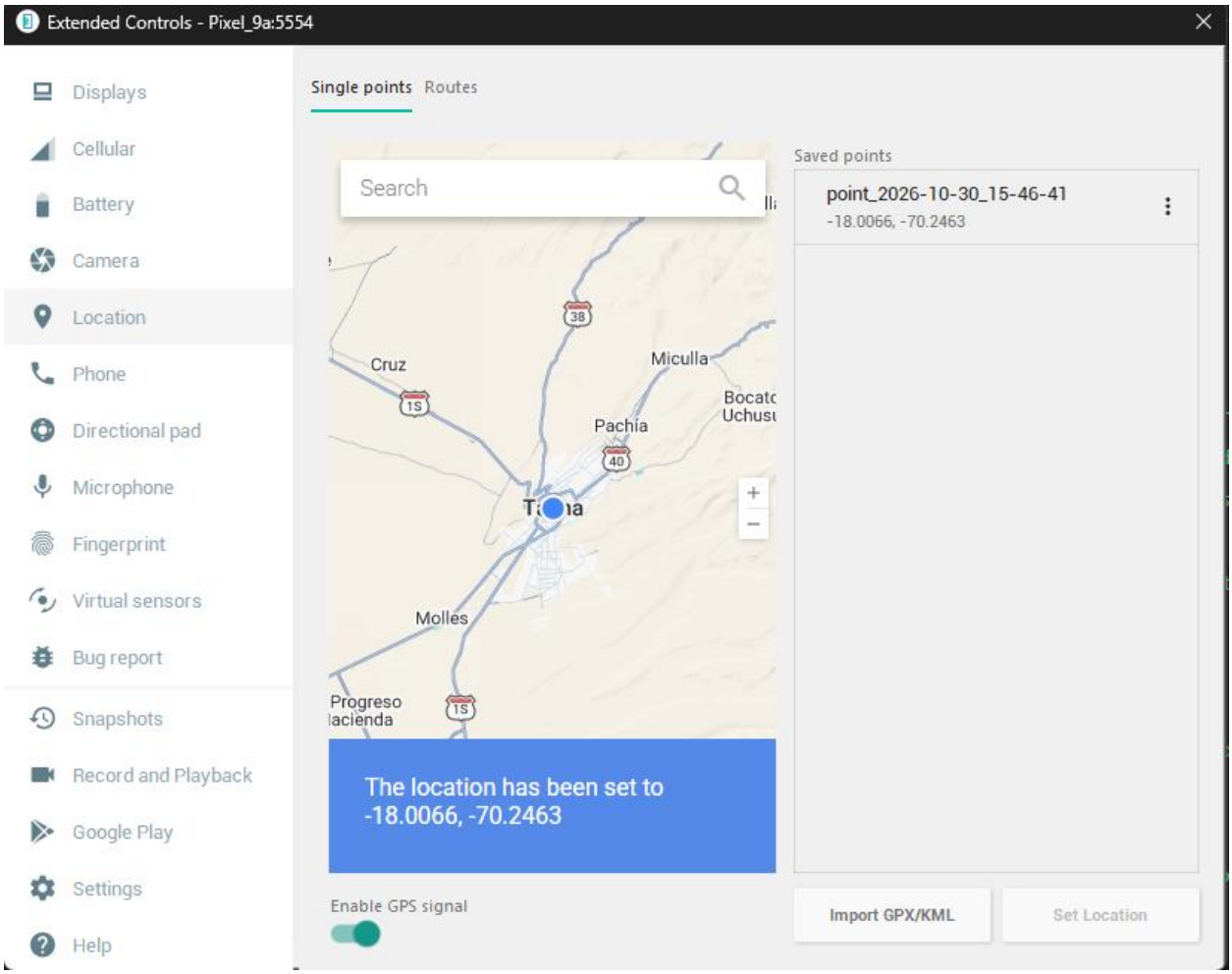


Figura 4. Posición fijada en los controles extendidos del emulador.

Paso 5 Coordenadas en Entre Huellas

Añadí ACCESS_FINE_LOCATION junto a ACCESS_COARSE_LOCATION en AndroidManifest.xml, compilé e instalé el APK y activé el permiso preciso de primer plano para la comprobación en el emulador. La fuente conserva LocationAccuracy.medium. Ingresé en Explorar, solicité la ubicación actual, apliqué la ubicación y abrí Mapa.

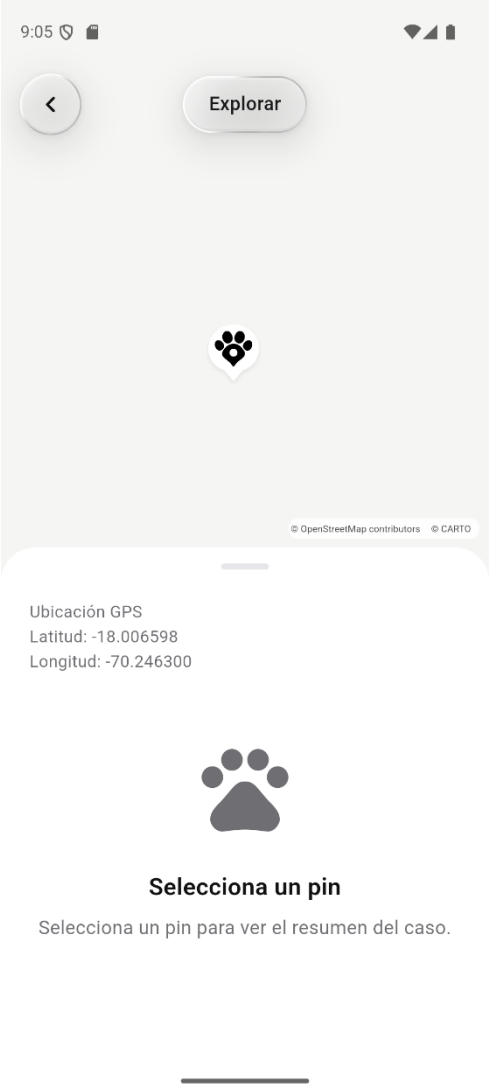


Figura 5. Mapa móvil con las coordenadas recibidas del dispositivo.

La app muestra latitud -18.006598 y longitud -70.246300, frente a -18.0066 y -70.2463 fijados en Extended Controls. La diferencia de latitud equivale aproximadamente a 0.22 m y queda dentro del margen de la lectura nativa. Se muestran los valores recibidos, sin sustituirlos por constantes ni corregirlos para que coincidan. El permiso preciso evita la alteración de coordenadas propia del permiso aproximado; la solicitud continúa siendo de precisión media.

Punto 2 Ubicación con consumo controlado

Implementé el seguimiento de ubicación en Explorar → Mapa de Entre Huellas. La biblioteca GPS permanece en DeviceLocationSource; la pantalla consume el repositorio mediante un caso de uso y un ViewModel. Este desarrollo corresponde exclusivamente al punto 2 del examen tipo 1.

Tarea 1 Precisión media

Configuré LocationAccuracy.medium, sin solicitar precisión máxima. Android declara ACCESS_COARSE_LOCATION y ACCESS_FINE_LOCATION; el permiso preciso no cambia la precisión media solicitada por la aplicación.

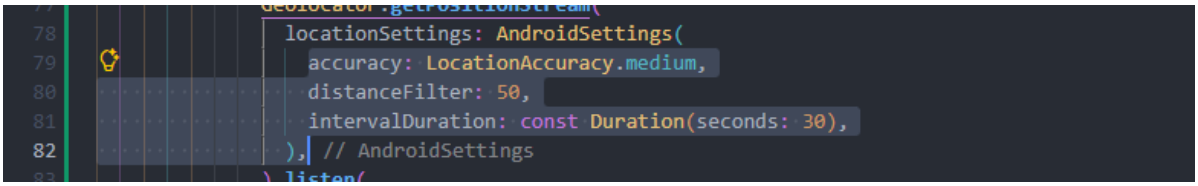


Figura 1. DeviceLocationSource, líneas 79–81: precisión media y límites de la solicitud nativa.

Tarea 2 Actualizaciones limitadas

La solicitud nativa usa intervalDuration de 30 segundos y distanceFilter de 50 metros. Además, cada nueva lectura se compara con la última aceptada: se descarta si transcurrieron menos de 30 segundos O si el desplazamiento fue menor de 50 metros. Así, ambas condiciones deben cumplirse para actualizar. Un Stopwatch mide el tiempo transcurrido sin depender de cambios en el reloj del sistema.



Figura 2. DeviceLocationSource, líneas 90–100: comprobación conjunta de tiempo y distancia.

Tarea 3 Tiempo agotado tras diez segundos

Después de verificar el servicio y el permiso, inicio un temporizador de 10 segundos para la primera lectura. Si vence, cancelo la suscripción GPS y propago un TimeoutException. Si llega una lectura, cancelo el temporizador. Permanecer quieto después de la primera lectura no genera falsos tiempos agotados.

```
67          if (cancelled) return;
68          // Timeout only for the first reading: being stationary is not an error.
69          firstReading = Timer(const Duration(seconds: 10), () async {
70            await stop();
71            if (!output.isClosed) {
72              output.addError(TimeoutException('No location in 10 seconds'));
73              unawaited(output.close());
74            }
75          }); // Timer
76          gps =
77              Geolocator.getPositionStream(
```

Figura 3. DeviceLocationSource, líneas 69–75: plazo inicial de 10 segundos y cancelación.

CurrentLocationRepositoryImpl transforma ese error en ExploreFailureKind.locationTimeout. ExploreMapScreen muestra exactamente «Tiempo agotado» en el toast superior compartido. Cuando falla el seguimiento, el mapa recibe una ubicación nula; no reutiliza una lectura antigua como ubicación actual válida.

```
64    final vm = ref.read(exploreViewModelProvider.notifier);
65    final location = ref.watch(exploreMapLocationProvider);
66    final locationNotice = switch (location.error) {
67      ExploreFailure(kind: ExploreFailureKind.locationTimeout) =>
68        'Tiempo agotado',
69      ExploreFailure(kind: ExploreFailureKind.locationDenied) =>
70        'No tenemos permiso de ubicación.',
```

Figura 4. ExploreMapScreen, líneas 66–68: mensaje visible del timeout.

Comprobación local

Ejecuté 29 pruebas pertinentes con resultado satisfactorio. Verifican precisión media, cumplimiento simultáneo de los dos límites, timeout inicial, cancelación durante la espera de la primera lectura, descarte de coordenadas anteriores ante error y navegación del mapa en tamaño normal y pequeño. El análisis Flutter terminó sin incidencias. Estas pruebas verifican el punto 2; no presentan como desarrollado el punto 4 del examen.

Tarea 4 Detener el GPS al salir del mapa

Al abandonar ExploreMapScreen, dispose invoca stop del ViewModel. También se detiene el seguimiento al abrir una ficha o pasar la aplicación a segundo plano. stop cancela la suscripción y escribe «GPS detenido» después de finalizar la cancelación.



Figura 5. ExploreMapLocationViewModel, líneas 43–51: cancelación antes del log.

La cancelación llega a DeviceLocationSource mediante onCancel. Su limpieza cancela el temporizador de primera lectura y la suscripción nativa, y detiene el reloj de seguimiento.

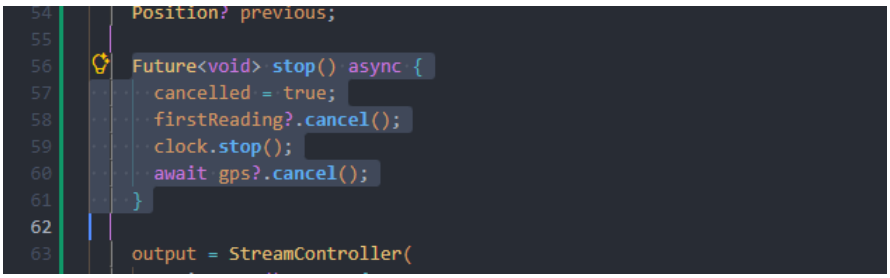


Figura 6. DeviceLocationSource, líneas 56–61: limpieza del temporizador y la escucha GPS.

Compilé e instalé el APK debug en emulator-5554. Abrí Explorar → Mapa, recibí una lectura y salí con Volver. El Logcat real registra «GPS detenido» y «Geolocator position updates stopped» con un milisegundo de diferencia. No cambié la posición del emulador ni apagué su GPS para esta evidencia.

```
10-30 21:24:00.227 E/FlutterGeolocator( 9385): Geolocator position updates started
10-30 21:24:51.950 I/flutter ( 9385): GPS detenido
10-30 21:24:51.951 E/FlutterGeolocator( 9385): Geolocator position updates stopped
```

Figura 7. Detalle del Logcat real de la consola tras abandonar el mapa. La captura completa se conserva como evidencia.

Punto 3 Mapa y precisión GPS

En Explorar reutilicé el mapa real con teselas OpenStreetMap y su atribución visible. La posición GPS se representa mediante un círculo cuyo radio en metros es accuracyMeters. Debajo se muestra Precisión ± N m. El círculo no utiliza el radio de búsqueda de casos.

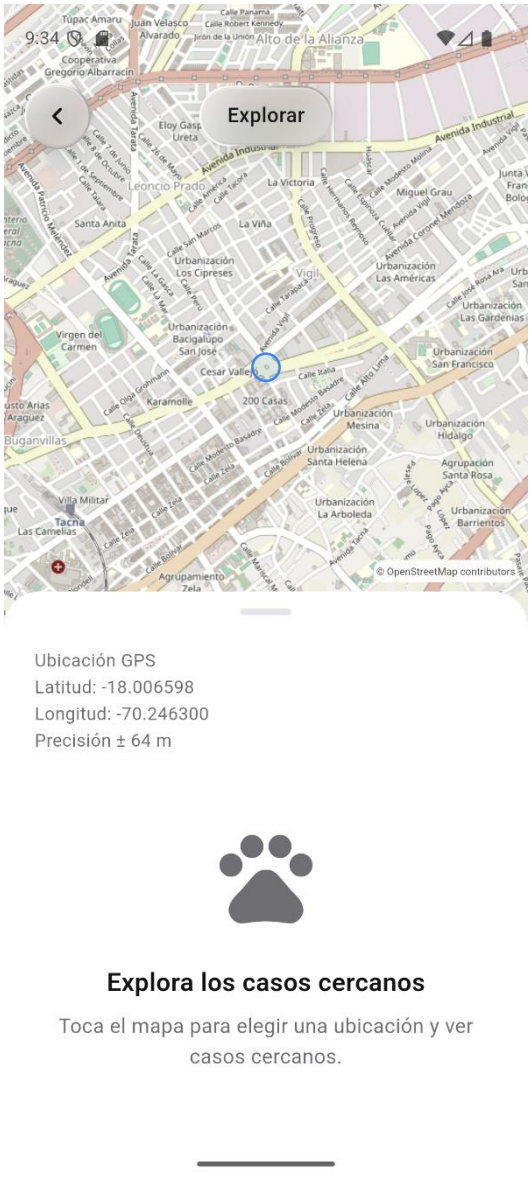


Figura 6. Captura real del emulador con círculo GPS, Precisión ± 64 m y atribución OpenStreetMap.

La lectura recibida fue -18.006598, -70.246300. El radio se conserva en metros; solo la etiqueta se redondea hacia arriba. Para apreciar el círculo se amplió el mapa mediante su gesto de zoom, sin cambiar las coordenadas ni la precisión recibidas.

Alternativa sin GPS

Apagué la ubicación del emulador y volví a abrir el mapa. El aviso Activa la ubicación del dispositivo explica el motivo. Toqué el mapa para elegir una ubicación manual; la aplicación muestra el pin oficial y sus coordenadas y aplica el centro a la búsqueda real. No muestra precisión GPS para el punto manual.

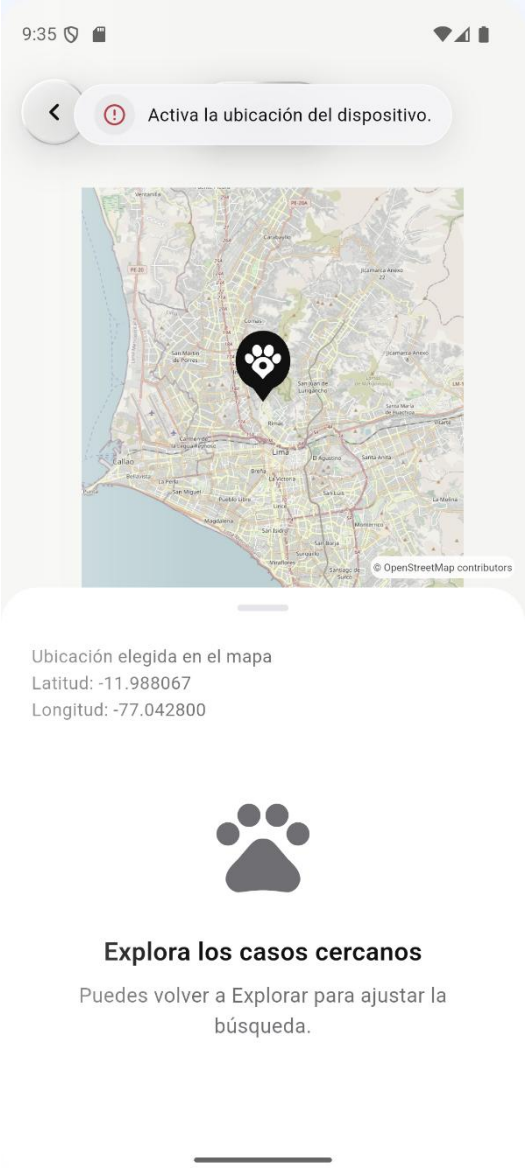


Figura 7. Captura real con ubicación desactivada y punto manual -11.988067, -77.042800.

Se comprobó con ADB que is-location-enabled devolvía false durante esta captura. Al terminar se restauró el GPS. La selección manual funciona también cuando el seguimiento informa permiso denegado o tiempo agotado; el motivo procede del estado de error y la lectura GPS anterior no se reutiliza.

Código del círculo y selección manual

ExploreCaseMap dibuja CircleMarker con radius: location.accuracyMeters y useRadiusInMeter: true. La pantalla consume exploreMapLocationProvider del punto 2; ante error entrega ubicación nula y habilita locationUnavailable.

```
showAttribution: false,
circles: [
  if (location != null)
    CircleMarker(
      point: LatLng(location.latitude, location.longitude),
      radius: location.accuracyMeters,
      useRadiusInMeter: true,
      color: const Color(0x223B82F6),
      borderColor: const Color(0xFF3B82F6),
      borderStrokeWidth: 2,
    ), // CircleMarker
],
```

Figura 8. Captura real de VS Code con el círculo en metros.

```
          ? 15000
          : math.max(75, location.accuracyMeters.ceil())),
onSelect:
    widget.locationUnavailable &&
        widget.onManualLocationSelected != null
    ? (latitude, longitude) {
        setState(() => _manualPoint = LatLng(latitude, longitude));
        widget.onManualLocationSelected?.call(latitude, longitude);
      }
    : null,
onMapTap: () => setState(() => _selectedCaseKey = null),
readOnly: !widget.locationUnavailable,
readOnlyNavigation: true,
fitRadius: true,
showCenterMarker: _manualPoint != null,
showRadius: false,
showAttribution: false,
circles: [
```

Figura 9. Captura real de VS Code con onSelect y la alternativa manual.

Validación: 43 pruebas previas de mapa y regresión y 30 pruebas con los puntos 2 y 3 integrados aprobadas, análisis sin incidencias y APK debug compilado e instalado en emulator-5554. Capturas del dispositivo y del editor reales; no se utilizaron renders de pruebas como evidencia del examen.